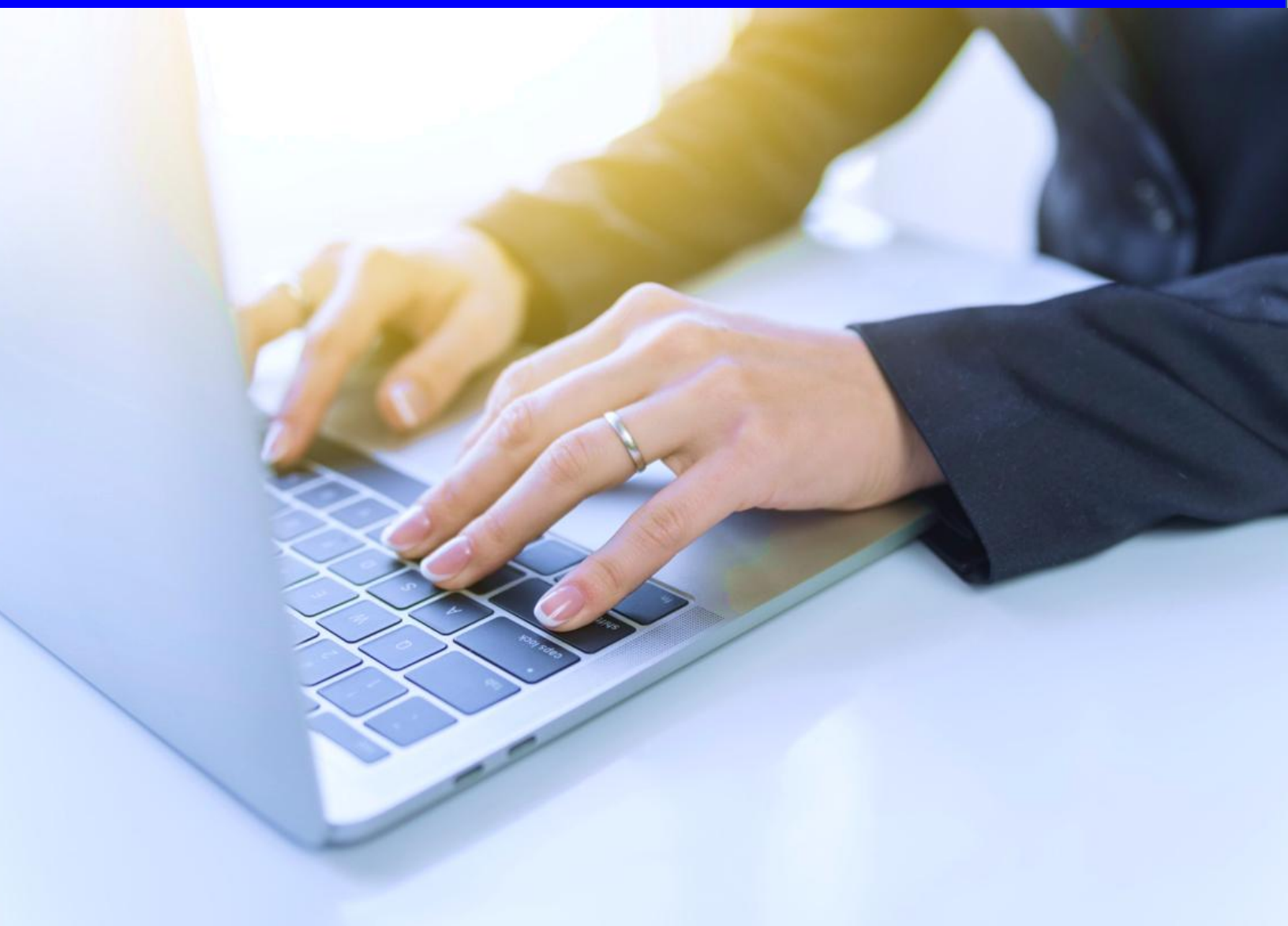




Nombre de la Asignatura:		CONSULTA DE DATOS
Unidad	Nº 1	CONSTRUYENDO CONSULTAS SIMPLES



Consideraciones Previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Organiza la lógica de negocio considerando restricciones del lenguaje, requisitos de la lógica de negocios, requisitos de información y sistemas de gestión de base de datos para solucionar los requerimientos de información planteados.
- Utiliza operadores lógicos, de comparación, matemáticos y cláusulas de ordenamiento para solucionar los requerimientos de información planteados.

SOBRE LAS FUENTES UTILIZADAS EN EL MATERIAL

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

En el siguiente Material de Estudio, se explicará la sentencia SELECT, que permite recuperar filas desde una o más tablas.

Se conocerán los operadores aritméticos: como suma, resta, multiplicación y división. También como es el tratamiento de los valores nulos dentro de la consulta.

Además, se abordarán los operadores relacionales, tales como: menor, mayor, menor igual, entre otros.

Para ello, se utilizarán varios ejemplos, detallando la lógica y resultado esperado.

Se invita al estudiante a seguir leyendo e interiorizarse de las posibilidades que ofrecen las consultas a las bases de datos.

Ideas fuerza

- Comando SELECT: Comando que permite obtener información de una o más tablas, y que será la base para la construcción de consultas a la base de datos durante todo este bimestre. Se explicará la sintaxis para el desarrollo de una consulta simple y compleja.
- Operadores relacionales: Los operadores relacionales comparan datos numéricos, de serie de caracteres o lógicos. El resultado de la comparación ya sea verdadero o falso, puede utilizarse para tomar una decisión referente a la consulta SELECT.

Índice

<i>Introducción</i>	3
<i>Ideas fuerza</i>	4
<i>Desarrollo</i>	6
Sentencia SELECT	6
Capacidades de la sentencia SELECT	6
Operadores Aritméticos	9
Operaciones con valores NULOS	11
Uso de Alias	12
Usando Operador de Concatenación	15
Valores Duplicados en las columnas	16
Ordenando las filas recuperadas	17
Restringiendo las Filas que son Seleccionadas	19
.....	19
Uso de Operadores	23
Operador BETWEEN	24
.....	25
.....	25
Operador IN	25
Operador LIKE	26
Operador NULL	28
<i>Conclusión</i>	30
<i>Referencias</i>	31

Desarrollo

Sentencia SELECT

La sentencia SELECT es una de las palabras clave más importante, si no la más importante, en SQL.

SELECT se utiliza para recuperar información de la base de datos. Imagine una base de datos que contiene información sobre películas como, por ejemplo, el título de la película, el género, el director, la fecha de estreno, el país, el idioma, la calificación de la película, la duración, entre otros.

Capacidades de la sentencia SELECT

Con una sentencia SELECT se pueden:

- Elegir las columnas de la tabla que retornará la consulta o query (Proyección).
- Elegir las filas de la tabla que retornará la query (Selección).
- Elegir los datos que están almacenados en diferentes tablas especificando la unión entre ellas, generalmente será por las columnas de tipo Claves Foráneas (Join).

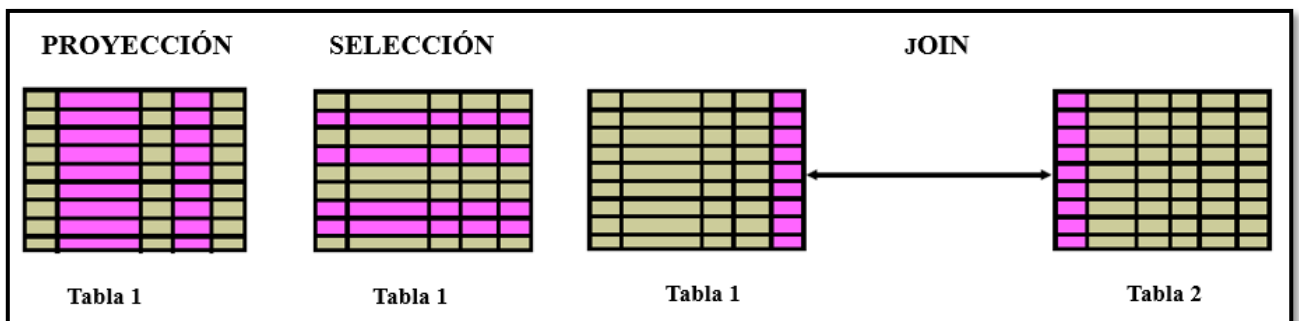


Imagen n°1
Fuente: Elaboración propia

Utilice SELECT permite mostrar los datos de las columnas de las tablas.

Sintaxis:

```
SELECT * | { [ DISTINCT ] columna/expresión [alias],...}  
FROM tabla  
[WHERE condición]  
[ORDER BY criterio_orden];
```

Imagen n°2

Fuente: Elaboración propia

Donde:

- SELECT es la cláusula que especifica una o más columnas que se desean mostrar.
- *: selecciona todas las columnas.
- DISTINCT: permite que los valores duplicados de las columnas se muestren una vez.
- Columna/expresión : selecciona el nombre de la columna o expresión.
- alias: da un nombre de salida a la columna seleccionada (nombre lógico).
- FROM tabla : especifica la tabla que contiene las columnas que se desean mostrar.
- WHERE condición : permite mostrar las filas que cumplan con la condición.
- ORDER BY criterio_orden : permite mostrar la información ordenada según el criterio de orden especificado.

Ejemplo: Se pueden seleccionar todas las columnas de datos de una tabla usando un asterisco (*) a continuación de la palabra SELECT.

Se seleccionan todos los campos de la tabla DEPARTMENTS, con todos sus registros (27 departamentos).

```
SELECT *
FROM departments;
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	1400
70	Public Relations	204	2700
80	Sales	145	2500
90	Executive	100	1700
100	Finance	108	1700
110	Accounting	205	1700
....			
....			
260	Recruiting		1700
270	Payroll		1700

27 filas seleccionadas

Imagen n°3
Fuente: Elaboración propia

¿Qué elementos se debería tener en cuenta si se pretende acceder a películas que solo estén disponibles en un determinado país y/o que compartan un mismo idioma?



Ejemplo: Se puede usar las sentencias SELECT para mostrar ciertas columnas de una tabla especificando el nombre de las columnas separadas por coma.

En la cláusula SELECT se especifica las columnas que contienen los datos que se desean mostrar y en el orden que se desea que aparezcan en la salida.

SELECT department_id, location_id FROM departments;		SELECT location_id, department_id FROM departments;	
DEPARTMENT_ID	LOCATION_ID	LOCATION_ID	DEPARTMENT_ID
-----	-----	-----	-----
10	1700	1700	10
20	1800	1800	20
30	1700	1700	30
40	2400	2400	40
50	1500	1500	50
60	1400	1400	60
70	2700	2700	70
.....			
.....			
270	1700	1700	270
27 filas seleccionadas		27 filas seleccionadas	

Imagen n°4
Fuente: Elaboración propia

Operadores Aritméticos

Para efectuar cálculos con los datos de las tablas se deben usar expresiones aritméticas.

- Una expresión aritmética puede contener nombre de columnas, constantes de valores numéricos y operadores aritméticos.
- Los operadores aritméticos se pueden usar en cualquier cláusula de una sentencia DML excepto en la cláusula FROM.

Los operadores que se pueden utilizar en una sentencia SQL son:

OPERADOR	DESCRIPCIÓN
+	Suma
-	Resta
*	Multiplicación
/	División

Imagen n°5
Fuente: Elaboración propia

El orden de prioridad de los operadores es la siguiente:

- La multiplicación y la división se ejecutan antes que la suma y resta.
- En los operadores con igual prioridad se ejecutan desde izquierda a derecha.
- Se puede evitar cumplir esa prioridad usando paréntesis; el interior de los paréntesis es lo que se ejecuta primero.
- Cuando una expresión aritmética se calcula sobre valores NULOS (NULL) el resultado de la expresión es siempre NULO (NULL).

Ejemplo: la sentencia muestra el apellido paterno, salario y salario más 300 dólares de todos los empleados que existen el tabla EMPLOYEES.

SELECT last_name, salary, salary + 300 FROM employees;			
LAST_NAME	SALARY	SALARY+300	
King	24000	24300	
Kochhar	17000	17300	
De Haan	17000	17300	
Hunold	9000	9300	
Ernst	6000	6300	
Austin	4800	5100	
Pataballa	4800	5100	
Lorentz	4200	4500	
Greenberg	12008	12308	
Faviet	9000	9300	
Chen	8200	8500	
Sciarra	7700	8000	
.....			
.....			
Gietz	8300	8600	
107 filas seleccionadas			

Imagen n°6
Fuente: Elaboración propia

Ejemplo: la sentencia muestra el apellido paterno, sueldo y el resultado de multiplicar el valor del sueldo por 12 y al valor obtenido de esa multiplicación sumarle 100 dólares. Por orden de prioridad de los operadores aritméticos primero se resuelve la multiplicación y posteriormente la suma.

```
SELECT last_name, salary, 12*salary+100
FROM employees;
```

LAST_NAME	SALARY	12*SALARY+100
King	24000	288100
Kochhar	17000	204100
De Haan	17000	204100
Hunold	9000	108100
Ernst	6000	72100
Austin	4800	57700
Pataballa	4800	57700
Lorentz	4200	50500
Greenberg	12008	144196
Faviet	9000	108100
.....		
.....		
Gietz	8300	99700

107 filas seleccionadas

Imagen n°7
Fuente: Elaboración propia

Operaciones con valores NULOS

Un Nulo en un valor no asignado, desconocido. Un Nulo no es lo mismo que un cero o espacio, ya que el cero es un valor y el espacio es un carácter.

Si el valor de cualquier columna en una expresión aritmética es Nulo el resultado es Nulo tal como se muestra en el ejemplo.

**SELECT last_name, salary, commission_pct , 12*salary*commission_pct
FROM employees;**

LAST_NAME	SALARY	COMMISSION_PCT	12*SALARY*COMMISSION_PCT
King	24000		
Kochhar	17000		
.....			
.....			
Cambrault	11000	0,3	39600
Zlotkey	10500	0,2	25200
Tucker	10000	0,3	36000
.....			
.....			
Gietz	8300		

107 filas seleccionadas

Imagen n°8

Fuente: Elaboración propia

Uso de Alias

Un Alias renombre una columna (su nombre) seleccionada cuando es mostrada. En otras palabras, ya no saldrá como cabecera, en el informe de salida, el nombre real de la columna que se está seleccionando sino que se mostrará como nombre el Alias dado a la columna real.

- Es útil cuando en una sentencia SQL se efectúan cálculos. Va inmediatamente a continuación del nombre de columna de la tabla.
- Opcionalmente se puede usar la palabra reservada AS entre el nombre de la columna y su Alias.
- Se requiere utilizar doble comilla si el Alias contiene espacios, caracteres especiales o si debe ser case-sensitive.

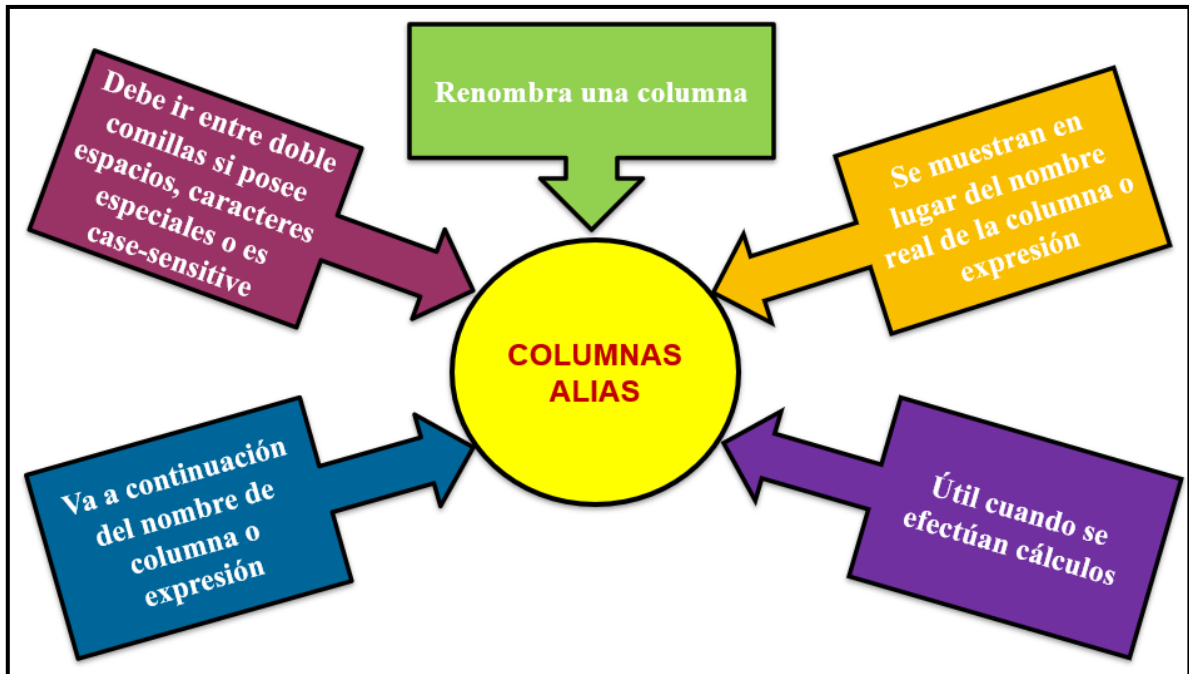


Imagen n°9
Fuente: Elaboración propia

Ejemplo: la sentencia muestra el apellido del empleado pero como cabecera en el informe ya no aparecerá el nombre de la columna que contiene ese dato en la tabla EMPLOYEES (last_name) sino que aparecerá NOMBRE. Además la sentencia muestra el porcentaje de comisión que tienen los empleados (columna commission_pct) con el nombre COMM.

```
SELECT last_name AS nombre , commission_pct comm
FROM employees;
```

NOMBRE	COMM
King	
Kochhar	
.....	
.....	
Russell	0,4
Partners	0,3
Errazuriz	0,3
Cambraut	0,3
.....	
.....	
Higgins	
Gietz	

107 filas seleccionadas

Imagen n°10

Fuente: Elaboración propia

Ejemplo: La sentencia del ejemplo muestra el apellido, el salario y el salario aumentado en 25% para cada empleado. Como cabecera en el informe aparecerá el nombre de la columna last_name, para el salario aparecerá Salario y para el resultado de la expresión que calcula el salario aumentado en 25% aparecerá con el nombre Salario Aumentado en 25%.

```
SELECT last_name, salary "Salario" , salary* 1.25 "Salario Aumentado en 25%"
FROM employees;
```

Nombre	Salario	Salario Aumentado en 25%
King	24000	30000
Kochhar	17000	21250
De Haan	17000	21250
Hunold	9000	11250
.....		
.....		
Higgins	12000	15000
Gietz	8300	10375

107 filas seleccionadas

Imagen n°11

Fuente: Elaboración propia

Usando Operador de Concatenación

El operador de concatenación || (dos barras verticales) une columnas y/o cadena de caracteres literales en otra columna. Esta columna resultante es una EXPRESIÓN de tipo caracter.

- Un literal es un Caracter (letra, espacio en blanco o símbolo especial), un Número o Fecha que está incluida en la sentencia SELECT.
- Las fechas y los literales de tipo caracteres que se desean concatenar deben ser escritos entre comillas simples.
- Cada cadena de caracteres es una salida por cada fila recuperada de la tabla.

Ejemplo: la sentencia muestra el apellido de los empleados unido al literal pertenece al departamento (incluidos espacios en blancos al comienzo y final) y a la identificación del departamento de cada empleado. La expresión que se obtiene de la concatenación se visualizará con el alias Detalle de Empleados.

```
SELECT last_name || ' pertenece al departamento ' ||  
       department_id "Detalle de Empleados"  
FROM employees;
```

```
Detalle de Empleados  
-----  
King pertenece al departamento 90  
Kochhar pertenece al departamento 90  
De Haan pertenece al departamento 90  
Hunold pertenece al departamento 60  
Ernst pertenece al departamento 60  
.....  
.....  
Higgins pertenece al departamento 110  
Gietz pertenece al departamento 110  
  
107 filas seleccionadas
```

Imagen n°12

Fuente: Elaboración propia

Ejemplo: La sentencia del ejemplo muestra el literal 'El salario mensual del empleado' unido al apellido de los empleados seguido del literal 'es de \$' unido al salario de cada empleado. La expresión que se obtiene de la concatenación será mostrada con el nombre de encabezado (alias) SALARIOS.

```
SELECT 'El salario mensual del empleado ' || last_name || ' es de $ ' || salary salarios
FROM employees;
```

```
SALARIOS
-----
El salario mensual del empleado King es de $ 24000
El salario mensual del empleado Kochhar es de $ 17000
El salario mensual del empleado De Haan es de $ 17000
El salario mensual del empleado Hunold es de $ 9000
El salario mensual del empleado Ernst es de $ 6000
El salario mensual del empleado Austin es de $ 4800
El salario mensual del empleado Pataballa es de $ 4800
El salario mensual del empleado Lorentz es de $ 4200
.....
El salario mensual del empleado Higgins es de $12008
El salario mensual del empleado Gietz es de $8300

107 filas seleccionadas
```

Imagen n°13
Fuente: Elaboración propia

Valores Duplicados en las columnas

Por defecto cuando en las consultas se obtienen los valores de las columnas que NO forman parte de la clave primaria se visualizan todas las filas incluyendo las duplicadas. Para eliminar las filas duplicadas en el resultado de la sentencia SELECT se debe incluir la palabra DISTINCT inmediatamente después de la palabra SELECT.

Por ejemplo, en la tabla EMPLOYEES un mismo código de departamento se puede repetir varias veces ya que en ese departamento pueden trabajar varios empleados. Por ello, en una sentencia SELECT normal si queremos mostrar solo los departamentos existentes se mostrarán todos los departamentos que existen en la tabla incluyendo los duplicados. Utilizando DISTINCT como parte de la sentencia SELECT se mostrarán una sola vez aquellos códigos de departamentos que se repiten en la tabla.

En el ejemplo, la sentencia de la izquierda se muestra TODOS los códigos de departamentos existentes en la tabla EMPLOYEES. En la sentencia de la derecha, utilizando DISTINCT se muestran los DIFERENTES códigos de departamentos existentes en la tabla EMPLOYEES, los departamentos que se repiten se muestran una sola vez.

SELECT department_id FROM employees;	SELECT DISTINCT department_id FROM employees;
<pre>DEPARTMENT_ID ----- 90 90 90 60 60 40 70 110 110 107 filas seleccionadas</pre>	<pre>DEPARTMENT_ID ----- 100 30 90 20 70 110 50 80 40 60 10 12 filas seleccionadas</pre>

Imagen n°14
Fuente: Elaboración propia

Ordenando las filas recuperadas

El orden de las filas recuperadas en una consulta es indefinido. Para ordenar las filas recuperadas se debe usar la cláusula ORDER BY.

- ASC: ordena en forma ascendente (defecto).
- DESC: ordena en forma descendente.

La cláusula ORDER BY debe ir al final de la sentencia SELECT. Como condición de ordenamiento se puede especificar una expresión, un alias o una columna.

En el ejemplo, las sentencias muestran el apellido, identificación del trabajo y fecha de contrato de los empleados ordena por fecha de contrato. La sentencia de la izquierda muestra la información ordenada en forma ascendente y la sentencia de la derecha la muestra ordenada en forma descendente.

SELECT last_name, job_id, hire_date FROM employees ORDER BY hire_date;			SELECT last_name, job_id, hire_date FROM employees ORDER BY hire_date DESC;		
LAST_NAME	JOB_ID	HIRE_DATE	LAST_NAME	JOB_ID	HIRE_DATE
-----	-----	-----	-----	-----	-----
King	AD_PRES	17/06/87	Kumar	SA_REP	21/04/00
Whalen	AD_ASST	17/09/87	Banda	SA_REP	21/04/00
Kochhar	AD_VP	21/09/89	Ande	SA_REP	24/03/00
Hunold	IT_PROG	03/01/90	Markle	ST_CLERK	08/03/00
Ernst	IT_PROG	21/05/91	Lee	SA_REP	23/02/00
.....					
.....					
Higgins	AC_MGR	07/06/94	Whalen	AD_ASST	17/09/87
Gietz	AC_ACCOUNT	07/06/94	King	AD_PRES	17/06/87
107 filas seleccionadas			107 filas seleccionadas		

Imagen n°15
Fuente: Elaboración propia

Ejemplo: La sentencia del ejemplo muestra la identificación, apellido y el salario multiplicado por 12 de cada empleado. Al resultado de la expresión de multiplicar el salario por 12 se le da el alias Salario Anual. La información se muestra ordenada en forma ascendente por el resultado de la multiplicación usando el alias.

SELECT employee_id, last_name, salary*12 "Salario Anual" FROM employees ORDER BY "Salario Anual" ;		
EMPLOYEE_ID	LAST_NAME	Salario Anual
-----	-----	-----
132	Olson	25200
128	Markle	26400
136	Philtanker	26400
135	Gee	28800
....		
....		
101	Kochhar	204000
100	King	288000
107 filas seleccionadas		

Imagen n°16
Fuente: Elaboración propia

Ejemplo: En el ejemplo, la sentencia muestra el apellido, identificación del departamento y salario de cada empleado. La información se mostrará ordenada en forma ascendente por departamento y por cada departamento se ordenará en forma descendente por salario.

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id, salary DESC;
```

LAST_NAME	DEPARTMENT_ID	SALARY
Whalen	10	4400
Hartstein	20	13000
Fay	20	6000
Raphaely	30	11000
Khoo	30	3100
Baida	30	2900
Tobias	30	2800
.....		
.....		
Urman	100	7800
Sciarra	100	7700
Popp	100	6900
Higgins	110	12000
Gietz	110	8300
Grant		7000

107 filas seleccionadas

Imagen n°17
Fuente: Elaboración propia

Restringiendo las Filas que son Seleccionadas

¿Qué se debe hacer si quisiéramos mostrar la identificación del empleado, su apellido, identificación del trabajo e identificación del departamento, pero sólo de aquellos empleados que trabajan en el departamento 90?



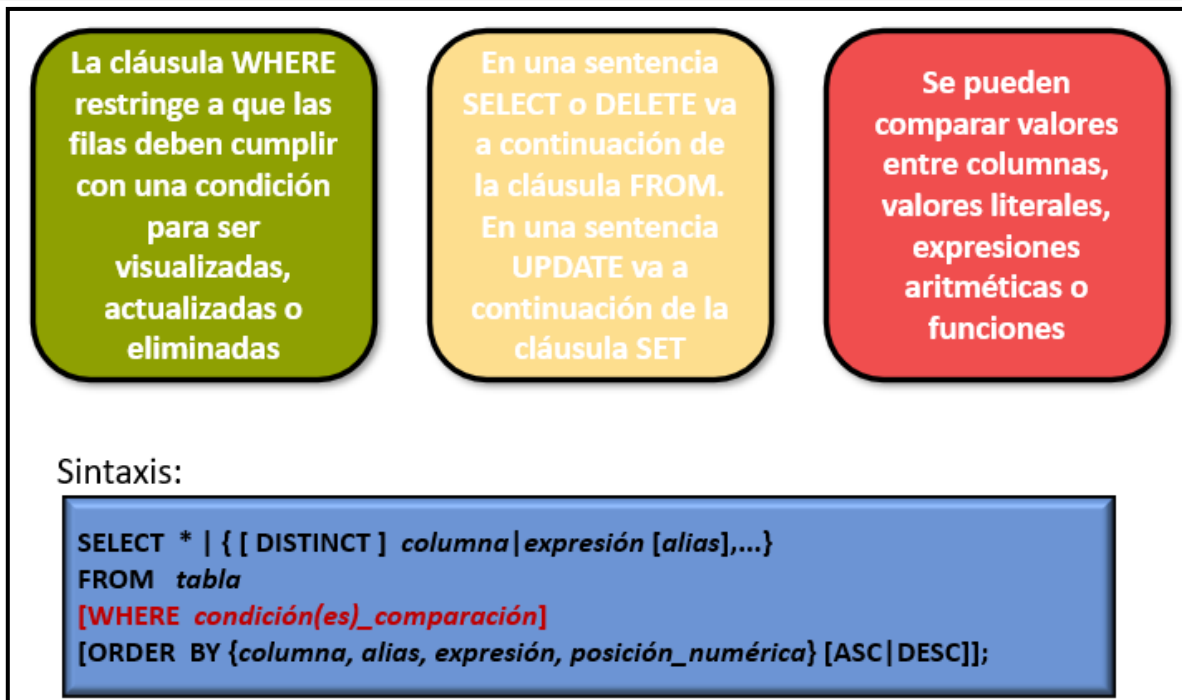


Imagen n°18
Fuente: Elaboración propia

Donde:

- **WHERE:** restringe la consulta a las filas que deben cumplir con la condición.
- **condición(es)_comparación:** está compuesta de nombre de columnas, expresiones, constantes y un operador de comparación.

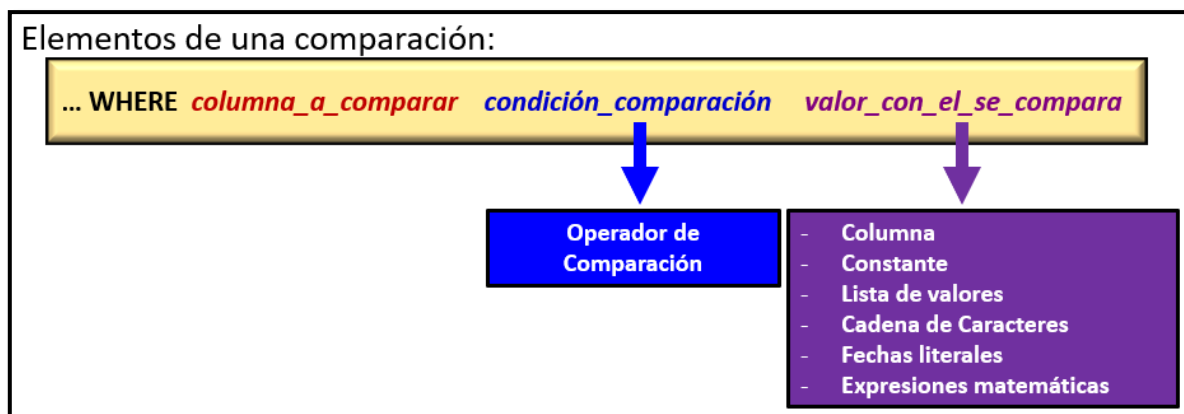


Imagen n°19
Fuente: Elaboración propia

La comparación consiste en tres elementos:

- Nombre de columna que servirá como restricción.

- Condición de comparación.
- Nombre de columna, constante o lista de valores con la que se desea comparar. También se puede usar cadena de caracteres y fechas literales:
 - Las cadenas de caracteres y fechas literales deben ir entre comillas simples.
 - Los valores caracteres son case-sensitive.
 - Los valores fechas son format-sensitive.

Ejemplo: En el ejemplo, y basados en la pregunta que se efectuó al comienzo, la sentencia muestra la identificación del empleado, su apellido, identificación del trabajo e identificación del trabajo de los empleados que trabajan en el departamento 90.

```
SELECT employee_id, last_name, job_id, department_id
FROM employees
WHERE department_id = 90;
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90

Imagen n°20
Fuente: Elaboración propia

Ejemplos: La primera sentencia muestra el apellido e identificación del departamento del o los empleados cuyo apellido sea igual a Whalen. Al efectuar una condición con un literal, este valor debe ir entre comillas simples y es CASE-SENSITIVE. Es decir, la Base de Datos buscará en la tabla EMPLOYEES todas las filas en las que en la columna last_name existe el mismo valor que se usa para comparar (de la misma forma en que está escrito).

Ejemplo:

```
SELECT last_name, job_id, department_id  
FROM employees  
WHERE last_name = 'Whalen';
```

	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	Whalen	AD_ASST	10

Ejemplo:

```
SELECT last_name, hire_date  
FROM employees  
WHERE hire_date > '01/03/2008';
```

	LAST_NAME	HIRE_DATE
1	Markle	08/03/2008
2	Ande	24/03/2008
3	Banda	21/04/2008
4	Kumar	21/04/2008

Imagen n°21

Fuente: Elaboración propia

En el segundo ejemplo, la sentencia muestra el apellido y fecha de contrato de los empleados cuya fecha de contrato sea mayor al 01 de Marzo del 2008. Como se compara con una fecha literal debe ir entre comillas simples y se compara en ese mismo formato.

Uso de Operadores

OPERADOR	DESCRIPCIÓN
=	Igual a
>	Mayor que
>=	Mayor o igual a
<	Menor que
<=	Menor o igual a
<>, !=, ^=	No igual o distinto a
BETWEEN ... AND ...	Entre dos valores (inclusivos)
IN (lista)	Valores que estén en la lista
LIKE	Valores que cumplan con una condición textual %: indica una serie cualquiera de caracteres _: indica un carácter cualquiera
IS NULL	Es un valor NULO

Imagen n°22

Fuente: Elaboración propia

Ejemplo Operador <=: En el ejemplo, la sentencia SELECT recupera el apellido paterno y el salario desde la tabla EMPLOYEES de cualquier empleado cuyo salario sea menor o igual a 2500. El valor explícito 2500 es comparado con el valor del salario de la columna SALARY de la tabla EMPLOYEES.

```
SELECT last_name, salary
FROM employees
WHERE salary <= 2500;
```

	LAST_NAME	SALARY
1	Colmenares	2500
2	Landry	2400
3	Markle	2200
4	Marlow	2500
5	Olson	2100
6	Gee	2400
7	Philtanker	2200
8	Patel	2500
9	Vargas	2500
10	Sullivan	2500
11	Perkins	2500

Imagen n°23

Fuente: Elaboración propia

Operador BETWEEN

- Muestra filas basadas en un rango de valores especificados.
- El rango que se especifica contiene un límite inferior y un límite superior.
- Los valores especificados con la condición BETWEEN son inclusive.

En el ejemplo, se recuperan las filas de los empleados cuyo salario esté entre los valores 2500 y 3500 inclusivos.


```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 2500 AND 3500;
```

Límite Inferior

Límite Superior

LAST_NAME	SALARY
1 Khoo	3100
2 Baida	2900
3 Tobias	2800
4 Himuro	2600
5 Colmenares	2500
6 Nayer	3200
7 Mikkilineni	2700
.....	
15 Patel	2500
16 Rajs	3500
17 Davies	3100
.....	
30 Walsh	3100
31 Feeney	3000
32 OConnell	2600
33 Grant	2600

Imagen n°24
Fuente: Elaboración propia

¿Cuál es la relevancia de considerar los elementos que posee una comparación



Operador IN

El operador IN Obtiene las filas cuyo valores se encuentren en la lista

Ejemplo: La sentencia SELECT del primer ejemplo recupera información de aquellos empleados cuyo apellido paterno sea Hartstein o Vargas.

En el segundo ejemplo, la sentencia SELECT recupera las filas de los empleados cuyos jefes tengan la identificación 100, 101 o 201.

```
SELECT employee_id, last_name,
       manager_id, department_id
FROM employees
WHERE last_name IN('Hartstein', 'Vargas');
```

EMPLOYEE_ID	LAST_NAME	MANAGER_ID	DEPARTMENT_ID
1	201 Hartstein	100	20
2	144 Vargas	124	50

```
SELECT employee_id, last_name,
       salary, manager_id
FROM employees
WHERE manager_id IN(100, 101, 201);
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
1	101 Kochhar	17000	100
2	102 De Haan	17000	100
3	114 Raphaely	11000	100
4	120 Weiss	8000	100
5	121 Fripp	8200	100
6	122 Kaufling	7900	100
7	123 Vollman	6500	100
8	124 Mourgos	5800	100
9	145 Russell	14000	100
10	146 Partners	13500	100

17	203 Mavris	6500	101
18	204 Baer	10000	101
19	205 Higgins	12008	101
20	202 Fay	6000	201

Imagen n°25
Fuente: Elaboración propia

Operador LIKE

El operador LIKE permite obtener filas cuyo valor en una columna cumpla una condición textual. Se utiliza especialmente con textos.

Utiliza una cadena de caracteres que pueden contener los siguientes símbolos:

- %: indica N caracteres.
- _: indica un carácter.

Ejemplo: En el ejemplo, se muestran los empleados cuyo apellido paterno comience con la letra S y le siga una cadena de caracteres cualquiera.

<pre>SELECT first_name FROM employees WHERE first_name LIKE 'S%';</pre>																													
	<table><tr><th></th><th>FIRST_NAME</th></tr><tr><td>1</td><td>Sundar</td></tr><tr><td>2</td><td>Shelli</td></tr><tr><td>3</td><td>Sarah</td></tr><tr><td>4</td><td>Shelley</td></tr><tr><td>5</td><td>Steven</td></tr><tr><td>6</td><td>Sundita</td></tr><tr><td>7</td><td>Steven</td></tr><tr><td>8</td><td>Susan</td></tr><tr><td>9</td><td>Samuel</td></tr><tr><td>10</td><td>Sarath</td></tr><tr><td>11</td><td>Stephen</td></tr><tr><td>12</td><td>Sigal</td></tr><tr><td>13</td><td>Shanta</td></tr></table>		FIRST_NAME	1	Sundar	2	Shelli	3	Sarah	4	Shelley	5	Steven	6	Sundita	7	Steven	8	Susan	9	Samuel	10	Sarath	11	Stephen	12	Sigal	13	Shanta
	FIRST_NAME																												
1	Sundar																												
2	Shelli																												
3	Sarah																												
4	Shelley																												
5	Steven																												
6	Sundita																												
7	Steven																												
8	Susan																												
9	Samuel																												
10	Sarath																												
11	Stephen																												
12	Sigal																												
13	Shanta																												

Imagen n°26
Fuente: Elaboración propia

En el primer ejemplo, la sentencia muestra las filas de aquellos empleados cuya fecha de contratación comienza con cualquier cadena de caracteres y los dos últimos deben ser 03.

El segundo ejemplo retorna las filas de los empleados cuyo apellido comience con un caracter (cualquiera) seguido de letra o (en minúscula) y a continuación la cadena de caracteres que sea.

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date LIKE '%03';
```

	LAST_NAME	HIRE_DATE
1	King	17/06/2003
2	Khoo	18/05/2003
3	Kaufling	01/05/2003
4	Ladwig	14/07/2003
5	Rajs	17/10/2003
6	Whalen	17/09/2003

```
SELECT last_name
FROM employees
WHERE last_name LIKE '_o%';
```

	LAST_NAME
1	Colmenares
2	Doran
3	Fox
4	Johnson
5	Jones
6	Kochhar
7	Lorentz
8	Mourgos
9	Popp
10	Rogers
11	Tobias
12	Vollman

Imagen n°27
Fuente: Elaboración propia

¿Cuál es la principal característica en común que comparten los Operadores? ¿En qué se sustenta la necesidad de que existan de distintos tipos?



Operador NULL

El operador NULL Evalúa valores Nulos. Esta condición incluye las condiciones IS NULL y la condición IS NOT NULL.

Un valor NULO es un valor no disponible, desconocido o inaplicable, por esta razón no se pueden evaluar con la condición igual a (=).

En el primer ejemplo, la sentencia SELECT retorna las filas de los empleados que no poseen jefe.

En el segundo ejemplo, la sentencia SELECT retorna las filas de los empleados que poseen porcentaje de comisión.

```
SELECT last_name, manager_id
FROM employees
WHERE manager_id IS NULL;
```

LAST_NAME	MANAGER_ID
King	(null)

```
SELECT last_name, job_id, commission_pct
FROM employees
WHERE commission_pct IS NOT NULL;
```

	LAST_NAME	JOB_ID	COMMISSION_PCT
1	Russell	SA_MAN	0,4
2	Partners	SA_MAN	0,3
3	Errazuriz	SA_MAN	0,3
4	Cambrault	SA_MAN	0,3
5	Zlotkey	SA_MAN	0,2
6	Tucker	SA_REP	0,3
7	Bernstein	SA_REP	0,25
8	Hall	SA_REP	0,25

33	Livingston	SA_REP	0,2
34	Grant	SA_REP	0,15
35	Johnson	SA_REP	0,1

Imagen n°28
Fuente: Elaboración propia

Conclusión

En el siguiente Material de Estudio, se explicó la sentencia SELECT, que permite recuperar filas desde una o más tablas.

Se conocieron los operadores aritméticos: como suma, resta, multiplicación y división. También como es el tratamiento de los valores nulos dentro de la consulta.

Además, se abordaron los operadores relacionales, tales como: menor, mayor, menor igual, entre otros.

Para ello, se utilizaron varios ejemplos, detallando la lógica y resultado esperado.

Referencias

Parte de la información ha sido extraída desde Oracle Academy. (s.f.).
(<https://myacademy.oracle.com>)



4 INSTITUCION
ACREDITADA
NIVEL AVANZADO
AÑOS Hasta octubre 2025



GESTIÓN INSTITUCIONAL Y DOCENCIA DE PREGRADO